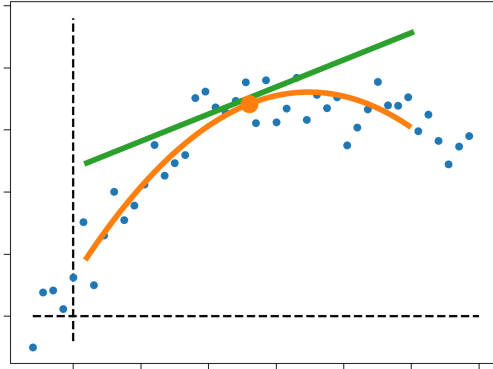




ERRATA TO

Hands-on Signal Analysis with Python

Author:
Thomas HASLWANTER

email:
office@thaslwander.at

Version: 1.1
August 24, 2026

© Thomas Haslwander, 2022

- p. 3, Eq. (1.1) should read:

The scalar product of two vectors **a** and **b** is defined as

$$\begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} \cdot \begin{pmatrix} b_x \\ b_y \\ b_z \end{pmatrix} = a_x b_x + a_y b_y + a_z b_z = |\mathbf{a}| * |\mathbf{b}| * \cos(\theta)$$

- p. 34, the last paragraph should begin as follows:

- To customize the `jupyter qtconsole` type
`jupyter console --generate-config`.

This creates the file `jupyter_qtconsole_config.py` in your Jupyter folder. (The Jupyter folder is the subfolder `/.` in your home directory.) In this file you find multiple options to configure your Qt

- p. 42, point “5” To edit files you have to type
`edit <fileName>`
- p. 60, code segment: not really a mistake, but possibly confusing — semicolons at the end of a Python line make no difference, and can be left away.
- p. 88 The command `scipy.integrate.cumtrapz` has been replaced with
`scipy.integrate.cumulative_trapezoid`
- p. 103, ‘Analyze EMG-data’: This exercise is wrongly listed here, it should only be included on p. 120 (Section 6.4.3.)
- p. 108, comment to line 35: “...is use ...” → “is used”
- p. 131, second code-segment:
All $(1 - \alpha)/2$ have to be replaced with $1 - \alpha/2$. So the correct code has to read

```
td = stats.t(df=4)
alpha = 5/100
tval =
td.ppf(1-alpha/2) # Result: tval = 2.78
# or in one line, now for 21 subjects
tval = stats.t(df=20).ppf(1-alpha/2) # Result: tval = 2.09
zval = stats.norm().ppf(1-alpha/2) # Result: zval = 1.96
```

- p. 131 box-content: The statement
... and the corresponding 95% CI is the area that contains the true mean value with a probability of 95%.
should be replaced with
... and the corresponding 95% CI should be interpreted in the following way: if the same sampling procedure were repeated 100 times from the same population, approximately 95 of the resulting intervals would be expected to contain the true population mean.
- p. 132, The following text should be added to the legend of Fig. 7.9:
Note: In the (rare) case that the true standard deviation σ is exactly known, then also the mean values are normally distributed!
- p. 139, Eq. (8.1) defining the correlation coefficient should read:

$$r = \sum_{i=0}^{n-1} \left(\frac{x_i - \bar{x}}{\sqrt{\sum_{j=0}^{n-1} (x_j - \bar{x})^2}} * \frac{y_i - \bar{y}}{\sqrt{\sum_{k=0}^{n-1} (y_k - \bar{y})^2}} \right)$$

- p. 150, Listing 8.2: If only the confidence intervals for the parameters of a linear regression are required, a simpler way to obtain them is with the newer package pingouin:

```
'''Calculation of confidence intervals for a line-fit'''

# Import the required packages
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
import pingouin as pg

# Generate the data
np.random.seed(123)
x = np.arange(100)
y = 30 + 0.4 * x + 5 * np.random.randn(len(x))

# Use a 'DataFrame' to contain the data, and a 'formula' to define the function
df = pd.DataFrame({'x': x, 'y': y})

# %% Solution with 'pingouin'
for alpha in [0.05, 0.001]:
    print(f'----- Result with pingouin, alpha={alpha:.3f} -----')
    results = pg.linear_regression(df.x, df.y, alpha=alpha)
    print(results)

# %% Solution with 'statsmodels'
print('\n ----- Results with statsmodels -----')
formula = 'y~x'
results = smf.ols(formula, data=df).fit()

# Show the results, with the default 95% confidence intervals
print(results.summary2())

# Address some of the fit-parameters
fit = {}
fit['parameters'] = results.params
fit['standard_error'] = results.bse
fit['ci_95'] = results.conf_int() # 95% confidence intervals
fit['ci_999'] = results.conf_int(alpha=0.001) # 99.9% confidence intervals

for key in [key for key in fit.keys() if key.startswith('ci')]:
    print(f'{key}: {fit[key]}')
```